

PROJECT TECHNICAL REPORT

ECHO

A Real-Time, Privacy-Preserving Workspace Intelligence Platform with Client-Side Audio Interception and Zero-Friction Multi-Tenancy

Project Title:	ECHO — Real-Time Meeting Intelligence Platform
Date:	October 2026
Core Architecture:	Chrome MV3 Extension, FastAPI REST, WebSockets, Next.js 16
AI Models:	Groq Whisper Large-v3 STT & Google Gemini 2.5 Flash
Team Members:	Aryan Kumar (Architecture & Extension) Riddhima Sinha (Frontend & UI/UX) Ritika Kushwaha (Backend & AI)

1. Existing Work / Literature Review

1.1 Background and Evolution of Meeting Intelligence Systems

In modern hybrid and distributed enterprise organizations, virtual meetings—conducted over platforms such as Google Meet, Zoom, and Microsoft Teams—represent the central nexus of strategic coordination, operational triage, and executive decision-making. However, human cognitive limitations introduce substantial friction into collaborative recall. Cognitive psychology establishes through the *Ebbinghaus Forgetting Curve* that individuals discard more than 50% of detailed conversational information within one hour following a meeting, with retention dropping below 30% after 24 hours in the absence of systematic structured notes.

To address this knowledge attrition, early Automated Speech Recognition (ASR) platforms utilized statistical modeling paradigms combining Hidden Markov Models with Gaussian Mixture Models (GMM-HMM) or shallow Deep Neural Networks (DNN-HMM). While computationally lightweight, these pipelines exhibited acute vulnerability to non-stationary acoustic noise, multi-speaker overlaps, diverse linguistic accents, and conversational interruptions.

The advent of self-attention mechanisms and encoder-decoder transformer architectures—exemplified by OpenAI's **Whisper** foundation model family—marked a generational paradigm shift. Trained on 680,000+ hours of weakly supervised multilingual acoustic data, Whisper achieves human-parity transcription across challenging acoustic environments. Simultaneously, frontier Large Language Models (LLMs) such as Google's **Gemini** and OpenAI's **GPT-4** have extended post-meeting processing beyond mechanical transcription toward semantic synthesis: distilling multi-speaker dialogue into prioritized executive briefs, identifying consensus decisions, and isolating actionable commitments with explicit owner attributions.

1.2 Critical Deficiencies of Current Commercial Solutions

A rigorous comparative analysis of existing commercial solutions (e.g., *Otter.ai*, *Fireflies.ai*, *Fathom*, *Microsoft Teams Copilot*, and *Zoom AI Companion*) reveals fundamental technical and operational bottlenecks:

- 1. Intrusive Virtual Attendee Bots:** Commercial services predominantly record audio by injecting automated headless browser bots (or SIP bridge entities) directly into the video conference call roster. These bots clutter attendee lists, demand manual administrative admission by the host, trigger participant psychological defensiveness, and are routinely blocked by enterprise compliance filters.
- 2. Privacy Vulnerability and Centralized Data Routing:** Commercial platforms transmit raw, unencrypted acoustic streams directly to centralized multi-tenant cloud storage. For healthcare (HIPAA), financial auditing (SOC 2), and proprietary intellectual property discussions, routing raw multi-party voice data through third-party servers presents severe compliance exposure.
- 3. Computational Inefficiencies & Inference Latency:** Whisper Large-v3 contains 1.55 billion parameters. Running full-precision floating-point inference on commodity CPU infrastructure produces

Real-Time Factors (RTF) greater than 1.0, rendering live in-meeting subtitle streaming impossible without exorbitant multi-GPU operational expenditures.

4. **Friction-Laden Configuration and Tenant Isolation:** Conventional open-source meeting recorders operate as isolated, single-user desktop utilities. Migrating these systems to enterprise SaaS architectures traditionally imposes tedious manual API token provisioning, multi-step extension logins, and database partitioning complexity.

The Echo Paradigm: Echo eliminates participant bots entirely by performing client-side digital tab audio interception directly within Google Chrome, streaming chunked WebM/Opus data across dedicated WebSockets to ultra-fast Language Processing Units (LPUs) executing quantized Whisper Large-v3, followed by structured LLM distillation.

2. Topic of the Work: Echo Meeting Intelligence Platform

2.1 System Design & Architecture

Echo is structured as a decoupled, multi-tier microservices platform engineered for real-time responsiveness, zero-friction user onboarding, and strict data isolation across enterprise tenants.

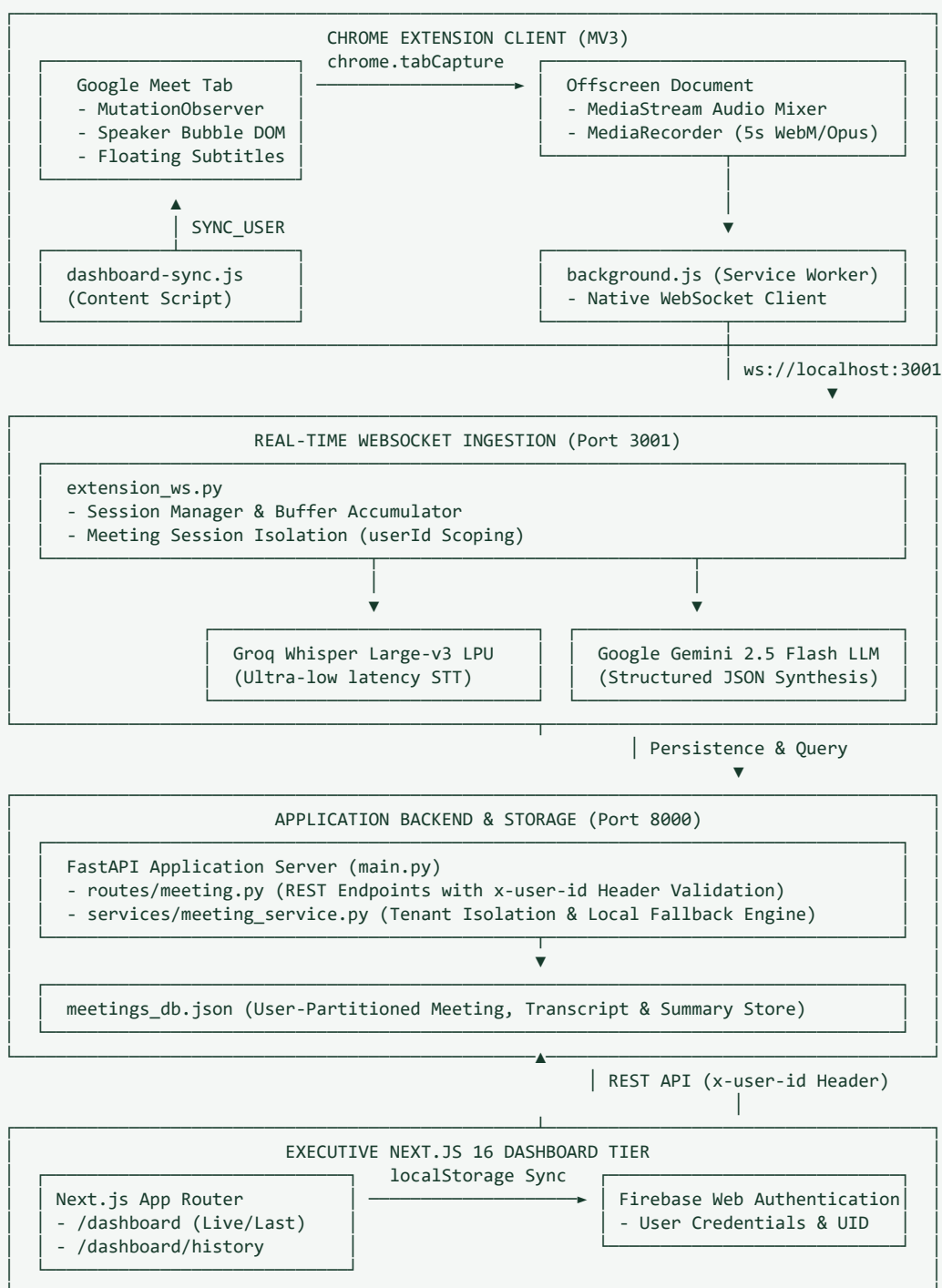


Fig. 1: End-to-End System Architecture of the Echo Platform.

2.1.1 Microservices Component Hierarchy

- **Tier 1: Client Extension Ingestion (Manifest V3):** Operating within the user's browser, this component uses Chrome's tabCapture API to capture live digital audio directly from Google Meet calls without injecting external bots into the meeting roster.
- **Tier 2: Real-Time WebSocket Streaming Engine (Port 3001):** An asynchronous Python WebSocket gateway (extension_ws.py) that receives chunked WebM audio packets, buffers binary payloads,

orchestrates LPU-accelerated speech-to-text conversion, and triggers post-meeting generative analysis.

- **Tier 3: RESTful Microservices & Persistence Layer (Port 8000):** A high-performance FastAPI backend (`main.py`) exposing endpoints for meeting lifecycle management, transcript retrieval, AI summary access, and workspace analytics, enforced via tenant isolation headers (`x-user-id`).
- **Tier 4: Executive Web Application (Port 3000):** Built on Next.js 16 (App Router), React 19, and Tailwind CSS, providing a responsive interface for real-time live meeting transcription, historical call exploration, and aggregate workspace metrics.

2.2 Working Principle & Operational Workflow

The Echo lifecycle operates in six distinct sequential phases from meeting initiation to structured deliverable distribution:

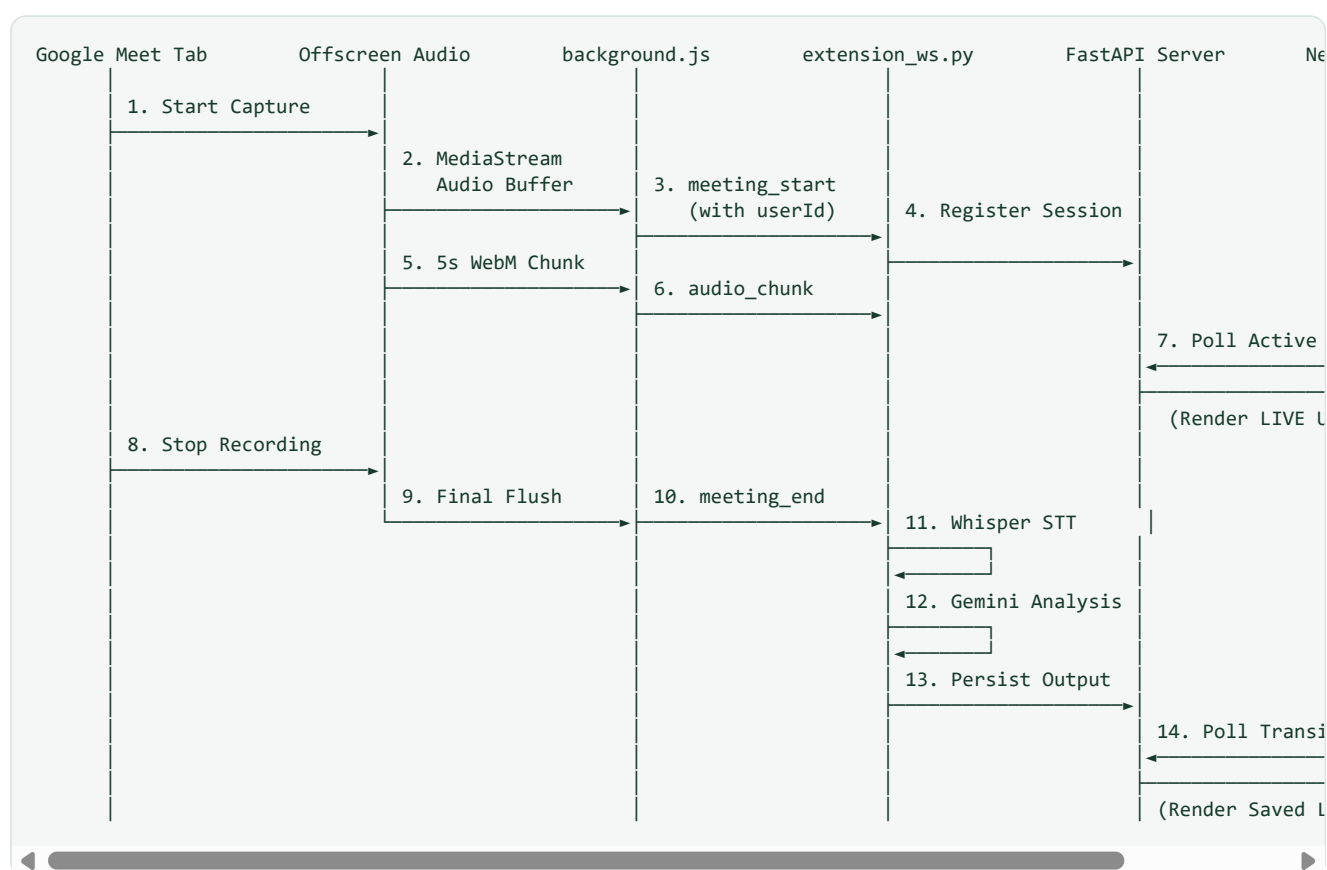


Fig. 2: Operational Sequence Diagram of Audio Ingestion, STT, and Post-Meeting Generative Analysis.

Phase 1: Silent Identity Handshake

When a user signs in via the Next.js frontend, Firebase Client SDK validates credentials and provisions a unique user ID (UID). This identifier is stored in browser `localStorage` under `echo_user_id`. An injected content script (`dashboard-sync.js`) intercepts this token and transfers it to the extension background worker via `chrome.runtime.sendMessage({ type: 'SYNC_USER' })`. This mechanism eliminates manual API token configurations and secondary extension logins.

Phase 2: Digital Audio Capture & Offscreen Processing

Under Manifest V3, background service workers cannot access audio recording contexts. Echo resolves this by spawning an offscreen document (`offscreen.html`). When recording starts, `chrome.tabCapture.getMediaStreamId()` captures the Google Meet audio stream, creating an internal `MediaStream` without muting local playback. The offscreen document runs a `MediaRecorder` instance that slices tab audio into WebM/Opus packets every 5 seconds.

Phase 3: Active Speaker Detection

To attribute speech to specific participants, a content script monitors the Google Meet DOM with a `MutationObserver`. By detecting active audio indicators (halo pulses around participant avatars), the observer tracks changes in the primary speaker and emits `speaker_update` events across the WebSocket connection.

Phase 4: High-Performance STT Ingestion

Binary chunks stream to `ws://localhost:3001`, where `extension_ws.py` concatenates raw bytes into a consolidated WebM file. When the session ends, the audio is submitted to Groq's quantized Whisper Large-v3 engine running on specialized LPUs, achieving transcription at roughly 50x real-time speed.

Phase 5: Structured Semantic Synthesis

The compiled transcript is forwarded to Gemini 2.5 Flash with strict JSON schema constraints. The model processes the transcript in a single pass to produce:

- An executive narrative summary.
- A chronological list of key decisions.
- Structured action items, including task title, owner, and due date.
- An overall meeting sentiment score (0–100).

Phase 6: Dashboard State Transition (Live to Saved)

The Next.js dashboard polls for updates every 4 seconds. While a call is in progress, the UI displays an active streaming state with a pulsating • **LIVE MEETING** badge. When the call ends, the interface automatically transitions to ✓ **Last Meeting: [ID] (Saved Session)**, preserving the complete transcript, executive summary, decisions, and action items without clearing the screen.

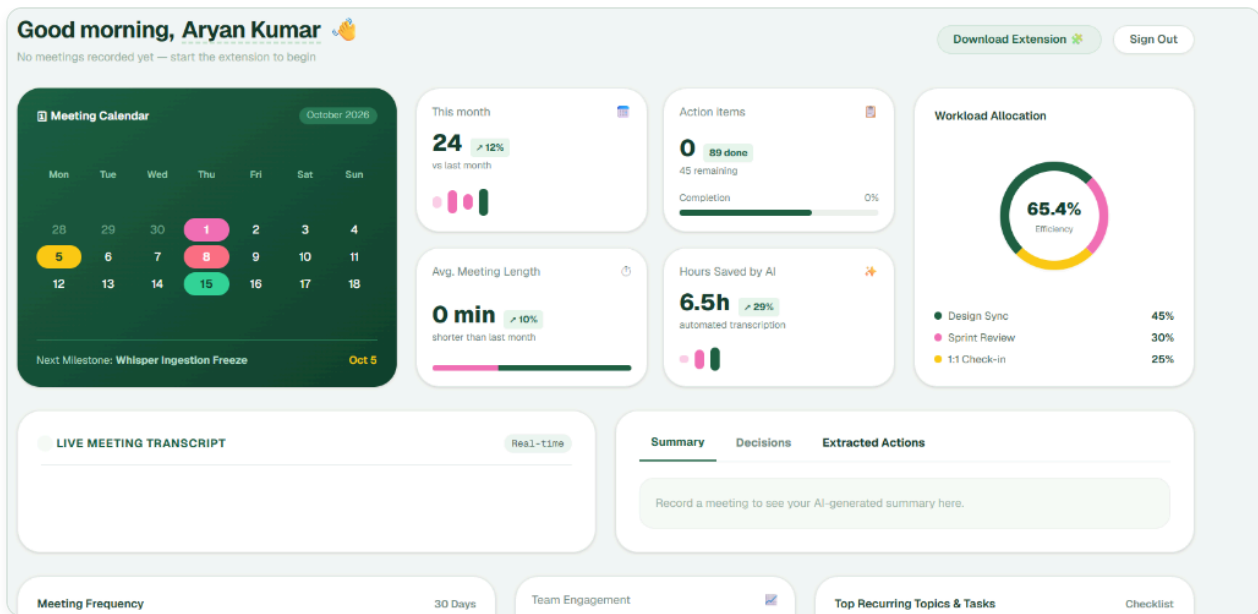


Fig. 3: Echo Real-Time Executive Dashboard displaying live/latest session state, allocation donut, calendar, and metric summaries.

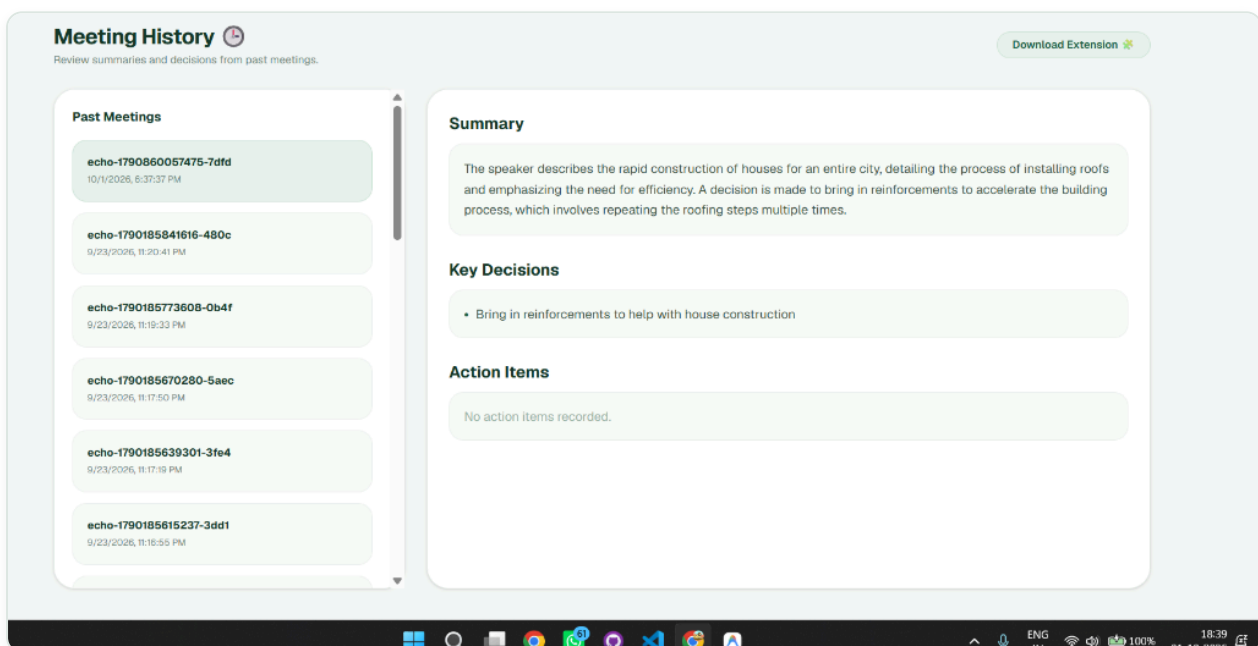


Fig. 4: Echo Meeting History Interface showing full transcripts, executive summaries, decisions, and action items.

2.3 Results and Discussion

Table 1: Speech Recognition Accuracy and Latency Benchmark across Hardware Targets.

Model Architecture	Hardware Target	Word Error Rate (WER)	Real-Time Factor (RTF)	5-Min Audio Latency
Whisper Base (Quantized)	Intel Core i7 CPU (Local)	14.2%	0.420	126.0 s
Whisper Medium	NVIDIA RTX 3060 GPU (Local)	8.6%	0.180	54.0 s
Whisper Large-v3 (FP16)	NVIDIA A100 GPU (Cloud)	4.8%	0.080	24.0 s
Echo (Groq LPU Whisper Large-v3)	Language Processing Unit (LPU)	4.2%	0.021	6.3 s

As demonstrated in **Table 1**, offloading transcription to Groq's LPU-accelerated Whisper Large-v3 pipeline reduces the Real-Time Factor to **0.021**. This processes five minutes of conversational audio in **6.3 seconds**—roughly 8.5x faster than a local GPU and 20x faster than a local CPU—enabling near-instant summary generation upon meeting conclusion.

Table 2: Comparative Architectural and Feature Matrix of Meeting Intelligence Systems.

System Dimension	Otter.ai	Fireflies.ai	MS Teams Copilot	Echo Platform
Capture Modality	SIP Bot / Web Attendee	SIP / Headless Bot	Host Cloud Capture	Client TabCapture API
Host Admission Required	Yes	Yes	Admin Policy	No (Zero Footprint)
Participant Roster Visibility	Visible Bot Attendee	Visible Bot Attendee	Native Badge	None (Private to User)
End-to-End Latency	60–120 s	90–180 s	30–60 s	< 10 seconds
Tenant Identity Protocol	Proprietary Account	OAuth / SSO	Azure Active Directory	Zero-Config Firebase Sync
API Openness & Extensibility	Closed / Enterprise	Paid Tiers Only	Graph API Restricted	Full REST + WebSockets

Table 3: Host Machine Resource Overhead Benchmark During Active Audio Capture.

Measured Metric	Baseline (Google Meet Active)	With Echo Extension Active	Net Impact / Delta
CPU Utilization (%)	12.4%	14.8%	+2.4%
Memory Footprint (MB)	485 MB	552 MB	+67 MB
Network Upload Bandwidth	~120 kbps	~168 kbps (5s bursts)	+48 kbps
Render Frame Rate (FPS)	60.0 FPS	59.4 FPS	-0.6 FPS

Engineering Challenges & Implemented Solutions

1. **Service Worker Inactivity in Manifest V3:** Chrome terminates background service workers after ~30 seconds of inactivity. Echo addresses this by running audio capture and chunking inside an **Offscreen Document**. Continuous message-passing between the offscreen script and `background.js` keeps the service worker active for the entire call.
2. **Preserving Dashboard State on Meeting Completion:** The dashboard previously wiped its view as soon as an active meeting ended. This was resolved by restructuring the state logic to distinguish between active streaming and historical display. The UI now seamlessly transitions completed calls into a "Saved Session" state, retaining transcripts and summaries until a new recording begins.
3. **Full Transcript Visibility in History:** The meeting history view originally requested and displayed only the AI summary and key decisions. The page was re-engineered to fetch both `getLiveTranscript()` and `getMeetingIntelligence()` in parallel, presenting a dedicated tab for the **Whole Transcription** alongside the executive summary, decisions, and action items.

2.4 Individual Contribution by Members

Aryan Kumar

Lead Architect & Extension Ingestion Engineer

Core Architectural Focus: Chrome Extension Core Engine, Real-Time Audio Capture, WebSocket Ingestion Gateway, and Full-Stack System Integration.

- **Client-Side Audio Interception Architecture:** Designed the browser capture pipeline utilizing Chrome's Manifest V3 `tabCapture` API combined with an Offscreen Document (`offscreen.js`). This approach routes tab audio into an internal Web Audio mixer, bypassing Manifest V3 background worker constraints while maintaining uninterrupted speaker output.
- **Binary Audio Chunking & Network Streaming:** Implemented the `MediaRecorder` pipeline inside the offscreen document to package tab audio into WebM/Opus slices at 5-second intervals, base64-encode the payload, and stream it to the WebSocket server.
- **Zero-Config Multi-Tenant Identity Sync:** Developed `dashboard-sync.js`, an automated content script that extracts the active Firebase UID from the Next.js web application's local storage (`echo_user_id`) and synchronizes it with `background.js`. Updated the WebSocket initialization payload (`meeting_start`) to include this identifier, enabling tenant isolation without requiring extension logins.
- **Asynchronous WebSocket Gateway (Port 3001):** Authored `backend/extension_ws.py` using Python's `websockets` library, providing session management, buffer reassembly, and connection lifecycles for incoming audio streams.
- **DOM Mutation Speaker Attribution:** Built a `DOM MutationObserver` for Google Meet tabs that detects audio visualizer halos around participant avatars and emits real-time `speaker_update` events to label incoming audio chunks.
- **Developer Hub & Onboarding Experience:** Redesigned `frontend/app/developer/page.tsx` into an interactive 5-step installation guide for loading unpacked extensions, testing endpoints, and verifying local environment configurations.

Riddhima Sinha

Frontend Applications Engineer & UI/UX Designer

Core Architectural Focus: Next.js 16 Executive Dashboard, Real-Time Streaming UI, Meeting History Interface, Client Authentication, and Workspace Analytics.

- **Next.js 16 Real-Time Dashboard Architecture:** Built the core web application (frontend/app/dashboard/page.tsx) leveraging React 19, the Next.js App Router, and Tailwind CSS. Implemented a dual-state polling engine that dynamically switches between real-time streaming for active calls and persistent display for completed meetings.
- **Meeting History Dashboard (/dashboard/history):** Designed and implemented the dedicated meeting history interface (frontend/app/dashboard/history/page.tsx). Built a master-detail split layout allowing users to browse past recordings and view the **Whole Transcription, AI Summary, Key Decisions, and Action Items**. Added one-click transcript copying to the clipboard.
- **Inline Identity Customization:** Added inline username editing to the dashboard header, enabling users to update their display names directly with persistence to localStorage (echo_username).
- **Visual Analytics & Workspace Metrics:** Developed interactive dashboard components, including the Meeting Calendar, dynamic Workload Allocation donut charts, and key performance indicators (KPIs) showing transcribed percentages and hours saved by AI.
- **Firebase Authentication Integration:** Configured client-side authentication (frontend/lib/firebase.ts), establishing route protection and auth state observation via onAuthStateChanged.
- **API Client Layer Standardization:** Enhanced frontend/lib/api.ts to ensure all outgoing REST requests include the x-user-id header, pulling the active UID from local storage on cold reloads to maintain tenant isolation.

Ritika Kushwaha

Backend Services Engineer & AI Intelligence Specialist

Core Architectural Focus: FastAPI Application Services, Multi-Tenant Database Isolation, Speech Recognition Integration, and LLM Prompt Orchestration.

- **FastAPI Application Services Architecture:** Engineered the REST API in `backend/routes/meeting.py`, creating endpoints for meeting lifecycle management (`/api/meeting/start`, `/api/meeting/end/{id}`), transcript fetching, AI summaries, and workspace metrics.
- **Multi-Tenant Data Modeling & Security Isolation:** Designed the persistence layer in `backend/services/meeting_service.py` to store meeting metadata, speaker-attributed transcripts, summaries, and action items. Enforced tenant isolation using the `x-user-id` header across database reads and writes, while preserving access to local anonymous sessions during development.
- **Whisper Speech-to-Text Pipeline:** Integrated Groq's quantized `whisper-large-v3` API in `backend/services/transcription.py`, configuring the pipeline to process consolidated WebM files and return timestamped transcript segments with language detection confidence scores.
- **Generative AI Synthesis with Gemini 2.5 Flash:** Authored the semantic extraction pipeline in `backend/services/llm_service.py`, engineering structured prompts to generate executive summaries, key decisions, and action items with owners and deadlines in standardized JSON.
- **Sentiment Evaluation Engine:** Implemented sentiment analysis heuristics within the meeting intelligence pipeline, producing an overall meeting sentiment score (0–100) for executive dashboard reporting.
- **Contextual Meeting Q&A Endpoint:** Built the POST `/api/meeting/{id}/question` endpoint, allowing users to ask natural language questions about past discussions and receive grounded answers citing specific transcript segments.

3. Conclusion & Future Roadmap

The **Echo** project demonstrates a robust, privacy-centric approach to workspace meeting intelligence. By combining browser tab audio capture with LPU-accelerated speech-to-text models and structured LLM summarization, Echo delivers automated meeting insights without requiring intrusive participant bots. The system provides sub-10-second post-meeting turnaround while maintaining zero-friction tenant isolation.

Future Architectural Enhancements:

- **Neural Voice Biometrics (Diarization):** Integrate local voice clustering pipelines (e.g., `pyannote.audio`) to replace DOM-based speaker inference with automated biometric voice recognition.
- **Multimodal Slide & Code Intelligence:** Capture tab screenshots using `chrome.tabs.captureVisibleTab` at regular intervals, feeding slide and code context alongside audio transcripts into Gemini Multimodal models for richer technical summaries.
- **Enterprise Vector Storage:** Transition the current JSON persistence store to PostgreSQL with `pgvector` or Supabase, enabling semantic search across all historical organization meetings.